

1 /*

2
3 **Exception:** An unwanted unexpected event that disturb normal flow of the program is called exception

4 --> It is highly recommended to handle exceptions.

5 --> The main objective of exception handling is normal termination of the program.

6
7 **Exception Handling:**

8 --> Exception handling does not mean repairing an exception.

9 --> We have to define alternative way to continue rest of the program normally.

10 --> This way of "Defining alternative" is nothing but exception handling.

11
12 --> Throwable acts as a root for exception hierarchy.

13 --> Throwable class contains the two child classes: Exception and Error

14
15 **Exception:** Most of the cases exceptions are caused by our program and these are recoverable.

16 **Error:** Most of the cases errors are not caused by our programs these are due to lack of system resources and these are not recoverable.

17
18 --> There are two types of Exceptions: Checked and Unchecked Exceptions

19
20 **Note:** Whether exception is checked or unchecked compulsory it should occur at runtime only.

21 --> There is no chance of occurring any exception at compile time.

22
23 **Examples of Exceptions:**

24 (1) RuntimeException

- 25 - ArithmeticException
- 26 - NullPointerException
- 27 - IndexOutOfBoundsException
- 28 - IllegalArgumentException
- 29 - ClassCastException
- 30 - IllegalStateException

31 (2) IOException

- 32 - EOFException
- 33 - FileNotFoundException

34 (3) SQLException

35 (4) ServletException

36 and Many more.....

37
38
39 **Exception Handling by try catch:**

40
41 --> It is highly recommended to handle exceptions.

42 --> In our program the code which may cause an exception is called risky code we have to place risky code inside try block

43 --> and the corresponding handling code inside catch block.

44
45
46 **Note:**

47 1. Within the try block if anywhere an exception raised then rest of

try block won't be executed even though we handled that exception.
-> Hence we have to place only risky code inside try and length
of the try block should be as less as possible.

```
*/  
  
package ExceptionHandling;  
  
public class EH1 {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
        // Without try catch  
        System.out.println(5 / 0);  
  
        /*  
        * with try catch  
        *  
        * try{  
        * System.out.println(5 / 0);  
        * } catch (ArithmeticException e) {  
        * System.out.println(10 / 2);  
        * }  
        *  
        */  
  
        System.out.println("World");  
    }  
}
```

```
1  /*
2
3  Various Methods to print Exception Information:
4
5  --> Throwable class defines the following methods to print exception
6  information to the console.
7
8  1. printStackTrace():
9      --> This method prints exception information in the following format:
10         Name of the exception: Description of the exception
11         Stack trace
12
13  2. toString():
14      --> This method prints exception information in the following format.
15         Name of the exception: Description of the exception
16
17  3. getMessage():
18      --> This method returns only description of the exception
19         Description
20
21  NOTE: Default exception handler internally uses printStackTrace() method
22  to print exception information to the console.
23
24  */
25
26  package ExceptionHandling;
27
28  public class EH2 {
29      public static void main(String[] args) {
30          int a[] = { 1, 2, 3, 4 };
31
32          try{
33              for (int i=1; i<=4;i++){
34                  System.out.println(a[i]);
35              }
36          } catch (ArrayIndexOutOfBoundsException e) {
37              System.out.println("Output of printStackTrace Method: ");
38              e.printStackTrace();
39
40              System.out.println("Output of toString Method: ");
41              // System.out.println(e.toString());
42              System.out.println(e);
43
44              System.out.println("Output of getMessage Method: ");
45              System.out.println(e.getMessage());
46          }
47      }
48  }
```

```
1  /*
2
3  try with multiple catch blocks:
4
5  --> The way of handling an exception is varied from exception to
6  exception
7      hence for every exception raise a separate catch block that is try
8      with multiple catch blocks is possible and
9      recommended to use.
10
11  */
12 package ExceptionHandling;
13
14 public class EH3 {
15     public static void main(String[] args) {
16         try {
17
18             //int c = 5 / 0;
19
20             int a[] = { 1, 2, 3, 4 };
21             for (int i = 1; i <= 4; i++) {
22                 System.out.println(a[i]);
23             }
24             // int c = 5 / 0;
25
26         } catch (ArithmeticException e) {
27             System.out.println(e);
28         } catch (ArrayIndexOutOfBoundsException e) {
29             System.out.println(e);
30         }
31         System.out.println("Last statement Executed");
32     }
33 }
```

```

1  /*
2
3  try with multiple catch blocks:
4
5  --> If try with multiple catch blocks presents then order of catch
6  blocks is very important.
7  --> It should be from child to parent.
8  --> By mistake if we are taking from parent to child then we will get
9  compile time error (C.E.) saying
10      "exception xxxx has already been caught".
11
12  */
13
14  package ExceptionHandling;
15
16  public class EH4 {
17      public static void main(String[] args) {
18          try {
19
20              // int c = 5 / 0;
21
22              int a[] = { 1, 2, 3, 4 };
23              for (int i = 1; i <= 4; i++) {
24                  System.out.println(a[i]);
25              }
26              // int c = 5 / 0;
27
28          } catch (Exception e) {
29              System.out.println(e);
30          }
31          // catch (ArrayIndexOutOfBoundsException e) { // Here we used
32          // child after parent so, C.E: Unreachable catch block
33          // for
34          // ArrayIndexOutOfBoundsException. It is already handled by the
35          // catch block for
36          // Exception
37          // System.out.println(e);
38          // }
39          System.out.println("Last statement Executed");
40      }
41  }

```

```
1  /*
```

```
3  Finally Block:
```

```
5      --> It is never recommended to take clean up code inside try block
6      because there is
7          no guarantee for the execution of every statement inside a try
8          block.
9      --> It is never recommended to place clean up code inside catch
10     block because
11         if there is no exception then catch block won't be executed.
12     --> We require some place to maintain clean up code which should be
13     executed always
14     irrespective of whether execution raised or not raised and
15     whether handled or not
16     handled such type of place is nothing but "finally block".
17     --> Hence, Main objective of finally block is to maintain cleanup
18     code.
```

```
13
14  Syntax:
```

```
15
16      try{
17          // risky code
18      } catch (X e){
19          // handling code
20      } finally {
21          // cleanup code
22      }
23  */
24
25  package ExceptionHandling;
26
27  public class EH5 {
28      public static void main(String[] args) {
29
30          try {
31              System.out.println("Try block executed");
32              //System.out.println(10/0);
33          } catch (ArithmeticException e) {
34              System.out.println("Catch block executed");
35              System.out.println(e);
36          } finally {
37              System.out.println("Finally block executed");
38          }
39
40      }
41  }
42
```

```
1  /*
2
3  Finally Block:
4
5      --> Even though return present in try or catch blocks, first finally
        will be executed.
6      --> And after that only return statement will be considered.
7      --> "finally" dominates "return statement".
8
9  */
10
11 package ExceptionHandling;
12
13 public class EH6 {
14     public static void main(String[] args) {
15         try{
16             System.out.println("try block executed");
17             return;
18         } catch (ArithmeticException e) {
19             System.out.println("catch block executed");
20         } finally {
21             System.out.println("finally block executed");
22         }
23     }
24 }
25
26
```

```
1  /*
2
3  Finally Block:
4
5      --> If return statment is present in try, catch and finally block
6      then finally block return statement will be considered.
7      --> There is only one situation where the finally block won't be
8      executed, i.e., whenever we are using System.exit(0) method.
9
10 */
11
12 package ExceptionHandling;
13
14 public class EH7 {
15     public static void main(String[] args) {
16         System.out.println(m1());
17     }
18
19     public static int m1() {
20         try{
21             System.out.println(5/0);
22             return 111;
23         } catch (ArithmeticException e) {
24             return 222;
25         } finally {
26             System.out.println("finally executed");
27             return 333;
28         }
29     }
30 }
```

```

1  /*
2
3  throw Statement:
4
5      --> To create exception object explicitly, one can use throw
6          statement.
7      --> By using throw statement, we can hand over exception to the JVM
8          manually.
9      --> The result of following 2 examples will be exactly the same.
10
11  */
12
13  package ExceptionHandling;
14
15  /*
16      In this case creation of ArithmeticException object and handover to
17      the JVM will be performed
18      automatically by the main() method.
19
20  */
21
22  public class EH8 {
23      public static void main(String[] args) {
24          System.out.println(5 / 0);
25      }
26  }
27
28  /*
29      In this case we are creating exception object explicitly and
30      handover to the JVM manually.
31
32  */
33
34  class EH8_1 {
35      public static void main(String[] args) {
36          throw new ArithmeticException("/ by Zero by CVM");
37      }
38  }
39
40  /*
41      CASE 1:
42      If Exception object refers null then we will get
43      NullPointerException.
44
45  */
46
47  class EH8_2_C1 {
48
49      static ArithmeticException e = new ArithmeticException(); // R.E:
50      ArithmeticException
51
52      public static void main(String[] args) {
53          throw e;
54      }
55  }
56
57  class EH8_3_C1 {

```

```

50
51     static ArithmeticException e; // R.E.: NullPointerException
52
53     public static void main(String[] args) {
54         throw e;
55     }
56 }
57
58 /*
59     CASE 2:
60         After throw statement we can't take any statement directly
61         otherwise we will get C.E. saying
62         "Unreachable code"
63 */
64
65 class EH8_4_C2 {
66     public static void main(String[] args) {
67         System.out.println(5 / 0); // R.E. ArithmeticException
68         System.out.println("Statement 1");
69     }
70 }
71
72 class EH8_5_C2 {
73     public static void main(String[] args) {
74         throw new ArithmeticException("/ by zero by CVM");
75         // System.out.println("Statement 1"); // C.E. Unreachable code
76     }
77 }
78
79 /*
80     CASE 3:
81         We can use throw keyword only for Throwable types otherwise we
82         will get C.E. saying
83         "incompatible types"
84 */
85
86 class EH8_6_C3 {
87     public static void main(String[] args) {
88         // throw new EH8_6_C3(); // C.E.
89         // Exception in thread "main" java.lang.Error: Unresolved
90         // compilation problem:
91         // No exception of type EH8_6_C3 can be thrown; an exception
92         // type must be a subclass of Throwable
93     }
94 }
95
96 class EH8_7_C3 extends RuntimeException {
97     public static void main(String[] args) {
98         throw new EH8_7_C3(); // R.E.: Exception in thread "main"
99         ExceptionHandling.EH8_7_C3
100             // at
101             ExceptionHandling.EH8_7_C3.main(EH8.java:96
102             )
103     }
104 }

```

```
98     }
99
100    class EH8_8_C3 extends Throwable {
101        public static void main(String[] args) {
102            // throw new EH8_8_C3(); // C.E.: Unhandled Exception type
103            EH8_8_C3
104        }
105    }
106
107    class EH8_9_C3 extends Exception {
108        public static void main(String[] args) {
109            // throw new EH8_9_C3(); // C.E.: Unhandled Exception type
110            EH8_9_C3
111        }
112    }
```

```

1  /*
2      throws Statement:
3          --> In program if there is chance of raising checked exception
4              then, compulsory we should handle it,
5          --> Either by try.... catch or by throws keyword,
6          --> Otherwise we will get C.E.
7
8  */
9
10 package ExceptionHandling;
11
12
13 public class EH9 {
14     public static void main(String[] args) {
15         // Thread.sleep(3000); // C.E.: Unhandled exception type
16         // InterruptedException
17     }
18 }
19
20 /*
21     We can handle above C.E. by using following 2 ways.
22     1. try...catch
23     2. throws keyword
24 */
25
26 class EH9_1 {
27     public static void main(String[] args) {
28         try {
29             Thread.sleep(3000);
30             System.out.println("Statement 1");
31         } catch (InterruptedException e) {
32             System.out.println(e);
33         }
34     }
35 }
36
37 class EH9_2 {
38     public static void main(String[] args) throws InterruptedException {
39         Thread.sleep(3000);
40         System.out.println("Statement 1");
41     }
42 }
43
44 /*
45     1. Main objective of "throws keyword" is to delicate the responsibility
46         of exception handling to the caller method.
47     2. "throws" keyword required only for checked exceptions.
48     3. "throws" keyword required only to convience compiler.
49     4. Usage of "throws" keyword does not prevent abrnormal termination of
50         the program.
51     5. We can not use "throws keyword with classes and interfaces"
52 */

```

```

52
53 class EH9_3 {
54     public static void main(String[] args) throws InterruptedException {
55         m1();
56     }
57
58     static void m1() throws InterruptedException {
59         m2();
60     }
61
62     static void m2() throws InterruptedException {
63         m3();
64     }
65
66     static void m3() throws InterruptedException {
67         Thread.sleep(3000);
68         System.out.println("Statement 1");
69     }
70 }
71
72 // In above program, if we are removing at least one throws keyword then
73 // the program will give C.E.
74
75 /*
76     CASE 1:
77     We can use throws keyword only for Throwable types otherwise we
78     will get compile time error
79 */
80
81 class EH9_4_C1{
82     public static void main(String[] args) /* throws EH9_4_C1 */ {
83         // C.E.: No exception of type EH9_4_C1 can be thrown; an
84         // exception type must be
85         // a subclass of Throwable
86     }
87 }
88
89 class EH9_5_C1 extends Throwable /*(Exception, RuntimeException also
90 possible)*/ {
91     public static void main(String[] args) throws EH9_5_C1 {
92
93     }
94 }
95
96 /*
97     CASE 2:
98 */
99
100 class EH9_6_C2 {
101     public static void main(String[] args) /* throws Exception */ {
102         // throw new Exception(); // C.E.: Because Exception is
103         // partially checked class, hence requires Handling
104         // throw new Throwable(); // C.E.: Because Throwable is
105         // partially checked class, hence requires Handling

```

```

101         // Solution is main method throws Exception.
102     }
103 }
104
105 class EH9_7_C2 {
106     public static void main(String[] args) {
107         throw new Error(); // R.E.
108     }
109 }
110
111 /*
112     CASE 3:
113         In our program, if there is no chance of rising an exception
114         then we can't right catch block for that exception
115         otherwise we will get C.E. saying exception xxxx is never thrown
116         in body of corresponding try statement.
117         --> This rule is applicable only for "fully checked exception".
118 */
119
120 class EH9_8_C3 {
121     public static void main(String[] args) {
122         try{
123             System.out.println("Statement 1");
124         }catch(Exception e){ // Exception is partially checked
125             System.out.println(e);
126         }
127     }
128 }
129
130 class EH9_9_C3 {
131     public static void main(String[] args) {
132         try {
133             System.out.println("Statement 1");
134         } catch (ArithmeticException e) { // ArithmeticException is
135             unchecked exception
136             System.out.println(e);
137         }
138     }
139 }
140
141 class EH9_10_C3 {
142     public static void main(String[] args) {
143         // try {
144         //     System.out.println("Statement 1");
145         // } catch (java.io.IOException e) { // C.E.:
146         //     System.out.println(e);
147         // }
148     }
149 }
150
151 // IOException is fully checked, Hence C.E.
152 // Unreachable catch block for IOException. This exception is never
153 // thrown from the try statement body
154
155 class EH9_11_C3 {
156     public static void main(String[] args) {

```

```
152         //      try {
153         //          System.out.println("Statement 1");
154         //      } catch (InterruptedException e) { // C.E.
155         //          System.out.println(e);
156         //      }
157     }
158 }
159
160
161 // InterruptedException is fully checked, Hence C.E.
162 // Unreachable catch block for InterruptedException. This exception is
    never thrown from the try statement body
163
164 class EH9_12_C3 {
165     public static void main(String[] args) {
166         try {
167             System.out.println("Statement 1");
168         } catch (Error e) { // Error is unchecked.
169             System.out.println(e);
170         }
171     }
172 }
```

```

1  /*
2      Customized Exceptions (User defined Exceptions):
3
4      --> We can create our own exception to meet our programming
5          requirements.
6      --> Such type of exceptions are called customized exceptions.
7
8  Note: It is highly recommended to maintain our customized exceptions as
9  unchecked by extending RuntimeException.
10     --> We can catch any Throwable type including Errors also.
11
12  */
13
14  class TooYoungException extends RuntimeException {
15      TooYoungException(String s) {
16          super(s);
17      }
18  }
19
20  class TooOldException extends RuntimeException {
21      TooOldException(String s) {
22          super(s);
23      }
24  }
25
26  class MarriageB {
27      public static void main(String[] args) {
28          int age = 0;
29          try{
30              age = Integer.parseInt(args[0]);
31              if (age > 60) {
32                  throw new TooYoungException("Please wait some more
33                      time.....You will get best match");
34              } else if (age < 18) {
35                  throw new TooOldException("Your age already crossed...No
36                      chance of getting Married");
37              } else {
38                  System.out.println("You will get match details soon by
39                      mail");
40              }
41          } catch (ArrayIndexOutOfBoundsException e) {
42              System.out.println(e);
43          }
44      }
45  }

```